

HAR Lab3 - Human Action Recognition

Sofía Jimeno Lucía
International Master in Computer Vision
Universidade do Porto
Porto, Portugal
up202200682@up.pt

Abstract

This report presents a comprehensive methodology for recognizing human actions using the KTH Actions dataset. The dataset comprises 599 videos, categorized into six distinct actions performed by 25 subjects under four different scenarios. Two approaches for video classification have been explored: a Convolutional Neural Network (CNN) combined with a Recurrent Neural Network (RNN) and a transformer-based model. The CNN-RNN architecture achieved an accuracy of 71.33%, while the transformer model reached a higher accuracy of 83.33%. This report outlines the problem, describes the dataset, and details the technical approaches implemented to solve the problem.

I. INTRODUCTION

Human action recognition is a critical task in computer vision, with applications spanning surveillance, human-computer interaction, and video indexing. The challenge lies in accurately identifying complex actions performed under varying conditions.

The objective of this project is to develop a methodology for action recognition using deep learning techniques. I have implemented two approaches in order to find a good performance in the classification problem: a CNN-RNN hybrid model and a transformer-based model. Both methods have been evaluated to determine their efficacy in classifying actions within the KTH dataset.

II. DATASET DESCRIPTION

The KTH Actions dataset includes 599 video sequences of six human actions: boxing, hand clapping, hand waving, jogging, running, and walking. Each action is performed by 25 subjects in four different scenarios: outdoors (d1), outdoors with scale variation (d2), outdoors with different clothing (d3), and indoors (d4). Videos have a spatial resolution of 160x120 pixels and an average duration of four seconds, captured at 25 frames per second.

III. TECHNICAL APPROACH

A. Data preparation

The steps followed for the preparing the data have been:

- Capturing the frames of a video.
- Extracting frames from the videos until a maximum frame count is reached, specifically a max length of 40.

- Splitting data into train (75%) and test (25%), using a 15% of the training as validation.

B. Approach 1: CNN-RNN Architecture

The CNN-RNN model combines the strengths of Convolutional Neural Networks (CNNs) for spatial feature extraction and Recurrent Neural Networks (RNNs) for temporal sequence modeling. The architecture consists of:

- **CNN Layers:** Extract spatial features from each video frame using convolutional layers followed by max-pooling. For our case, I have used a pre-trained network from Keras Applications, specifically the InceptionV3 using the ImageNet weights, to extract meaningful features from the extracted frames. I have also tried another type but it didn't improve.
- **Flattening and Dense Layers:** Flatten the output from the CNN layers and pass it through fully connected dense layers to reduce dimensionality. Finally, the number of features or embedding lengths extracted have been 2048.
- **RNN Layers:** in this case Gated Recurrent Units (GRUs) was used to capture temporal dependencies across frames. This sequence model needs as an input two arrays: features extracted and masks, where this last one contains a bunch of booleans denoting if a timestep is masked with padding or not.
- **Output Layer:** A softmax layer to classify the actions into one of the six categories.

The model was trained using the Adam optimizer and categorical cross-entropy loss function, achieving an accuracy of 71.33% on the test set.

C. Approach 2: Transformer-Based Model

Transformers, originally designed for natural language processing, have shown promising results in video classification due to their ability to handle long-range dependencies. The architecture is as follows:

- **CNN for extracting features:** As in the case above, the two first steps are the same. However, the pre-trained network used here was DenseNet121 (from Keras Applications) and the length of the features have been 1024.
- **Transformer Encoder:** Self-attention layers form the basic blocks of a Transformer are order-agnostic. Since videos

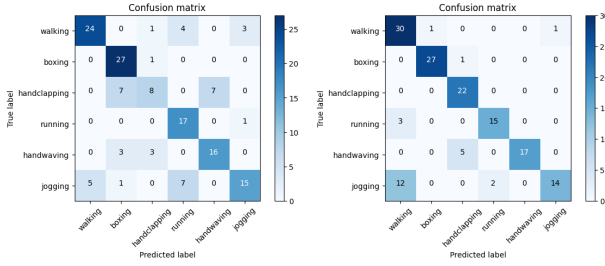


Fig. 1. Confusion Matrix for CNN-RNN model and Transformer-based model respectively.

are ordered sequences of frames, we need our Transformer model to take into account order information. This is done via positional encoding. Basically, positions of the frames present inside videos are embed with an Embedding layer. Then these positional embeddings are add to the precomputed CNN feature maps. The output is passed as input of multi-head attention layer.

- **Classification Head:** A global average pooling layer followed by a dense layer with a softmax activation function classifies the actions.

The transformer model uses the Adam optimizer with a learning rate scheduler. This model achieved a higher accuracy of 83.33% on the test set, outperforming the CNN-RNN architecture.

IV. RESULTS

A. Approach 1

The performance of the CNN-RNN classifier was evaluated over 30 epochs, with the resulting accuracy and loss trends displayed in Figure 2 and Figure 3, respectively. The model's performance on the training set was significantly higher than on the validation set, suggesting potential overfitting. However, at the end, in the test set the model got a 71.33% of accuracy for the classification.

The confusion matrix in Figure 1 provides a detailed of the classifier's performance across the six activity classes. The CNN-RNN classifier demonstrated a good performance in distinguishing between different physical activities, particularly with boxing and running. However, the confusion matrix highlights areas for improvement, particularly in better distinguishing between handclapping and handwaving, as well as reducing the misclassification between walking and jogging.

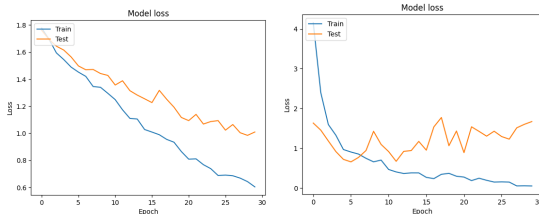


Fig. 3. Loss history plot for CNN-RNN model and Transformer-based model respectively.

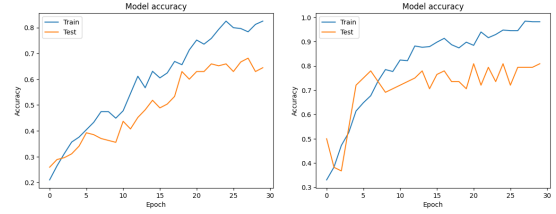


Fig. 2. Accuracy history plot for CNN-RNN model and Transformer-based model respectively.

In Figure 4, there are two examples, the first one from the boxing folder, where the model predictions are:

- boxing: 73.51%
- handwaving: 14.75%
- jogging: 6.23%
- running: 3.02%
- handclapping: 1.78%
- walking: 0.71%

And the second one from handwaving folder, where the model predictions are:

- handwaving: 79.40%
- boxing: 16.78%
- handclapping: 1.60%
- running: 0.85%
- jogging: 0.73%
- walking: 0.64%

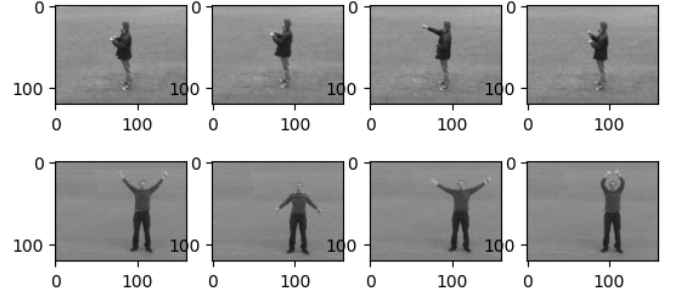


Fig. 4. Video examples of test dataset for Transformed-based model

B. Approach 2

As for the CNN-RNN classifier, the performance of the Transformed-based classifier was evaluated over 30 epochs, with the resulting accuracy and loss trends displayed in Figure 2 and Figure 3, respectively. The model's performance on the test set the model got a 83.33% of accuracy for the classification.

Again, as the Approach 1, the confusion matrix in Figure 1 shows that classifier demonstrated a good performance in distinguishing between different physical activities. However, there are some missclassification between handclapping and handwaving, and it has a lot difficulties classifying walking and jogging.

In Figure 5, there are two examples, the first one from the handclapping folder, where the model predictions are:

- handclapping: 100.00%
- walking: 0.00%
- boxing: 0.00%
- handwaving: 0.00%
- jogging: 0.00%
- running: 0.00%

And the second one from jogging folder, where the model predictions are:

- walking: 99.11%
- jogging: 0.88%
- running: 0.01%
- handclapping: 0.00%
- boxing: 0.00%
- handwaving: 0.00%

Showing one of the missclassifications mention before.

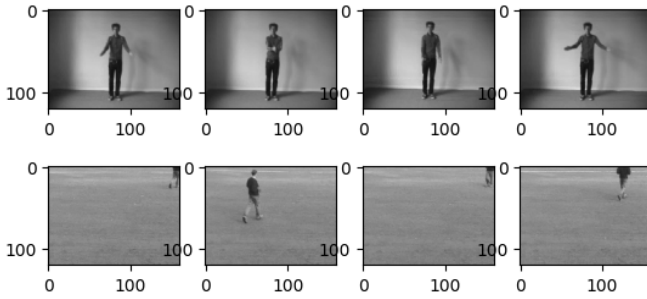


Fig. 5. Video examples of test dataset for CNN-RNN model

V. CONCLUSION

In this report, we have demonstrated the application of two advanced deep learning approaches to the problem of human action recognition using the KTH Actions dataset. The transformer-based model outperformed the CNN-RNN architecture, highlighting its superior capability in capturing temporal dependencies in video data. However, the performance of the models are very optimal, maybe because the number of videos is not enough and, maybe, because I have reduced the number of frames.

Future work could explore further optimizations and hybrid models, using regularization techniques or additional data augmentation to enhance accuracy and robustness in action recognition tasks.

REFERENCES

- [1] https://keras.io/examples/vision/video_classification/
- [2] https://keras.io/examples/vision/video_transformers/
- [3] <https://keras.io/api/applications/>https://keras.io/api/layers/recurrent_layers/gru/
- [4] <https://keras.io/api/applications/>